

Lab 2: Decision Tree Modeling and Improvement

Predicting Student Dropout and Academic Success

Group [TODO — GroupID]
[TODO — Course / instructor]

August 30, 2026

a. Group Introduction

[TODO — Group name and GroupID]

Table 1: Team members and contributions

Name	Student ID	Contribution	%
[TODO — Name A]	[TODO — ID]	Data collection & description, EDA, data preprocessing pipeline, final results aggregation, wrote Introduction / Dataset Description / Comparison / Conclusion	[TODO — %]
[TODO — Name B]	[TODO — ID]	Baseline model, tree/rule visualization tooling, wrote Baseline Model & Analysis of the Tree	[TODO — %]
[TODO — Name C]	[TODO — ID]	Cost-complexity pruning (Improvement 1), wrote section f.1	[TODO — %]
[TODO — Name D]	[TODO — ID]	Class imbalance handling (Improvement 2), wrote section f.2	[TODO — %]
[TODO — Name E]	[TODO — ID]	Feature selection for early warning (Improvement 3), slides, video, references, wrote section f.3	[TODO — %]

[TODO — Adjust the contribution text/percentages to match what each person actually did – cross-check against git log and progress/*.md before finalizing, not just this template.]

b. Introduction

b.1 Brief introduction to decision trees

A decision tree is a supervised learning model used for both classification and regression tasks. It represents the decision process as a tree structure: each *internal node* tests one feature (e.g. “Tuition fees up to date ≤ 0.5 ?”), each *branch* corresponds to the outcome of that test, and each *leaf node* assigns a predicted class.

Training a decision tree follows a **greedy, top-down, recursive-partitioning** strategy: at each node, the algorithm evaluates every feature and every possible split threshold, and picks the split that reduces class *impurity* the most, then repeats recursively on each resulting subset. Two impurity measures are standard:

- **Entropy / Information Gain** – used by the ID3 algorithm (?), measuring the information gained by a split.
- **Gini impurity** – used by CART (?), measuring the probability of misclassification if a label were drawn at random from the node’s class distribution.

The scikit-learn library used in this project implements an optimized variant of CART (?); it defaults to Gini but supports switching to entropy via the `criterion` parameter.

The **main strength** of a decision tree compared to “black-box” models (neural networks, complex ensembles) is **interpretability**: every path from the root to a leaf can be read directly as an IF (condition 1) AND (condition 2) ... THEN (label) rule, with no separate explanation technique (SHAP, LIME) required. Once nominal categories have been encoded, trees handle the resulting numerical and indicator features without feature scaling – both properties are exploited directly in Section c below.

The **main weakness** is a tendency to **overfit**: an unconstrained tree keeps splitting until almost every leaf is pure on the training set, effectively memorizing training noise, which yields very high training accuracy but a much lower test accuracy (??). This is exactly what is observed for the baseline model in Section d, and is the motivation for the pruning improvement in Section f.

b.2 Objective of the project

The project applies this theory to a real, socially meaningful problem: **predicting whether a student drops out, remains enrolled longer than expected, or graduates on time**, using real data from 4,424 university students in Portugal. Concretely, the group:

1. Builds and evaluates an unconstrained baseline decision tree, measuring performance with the full set of metrics appropriate for a 3-class classification problem (Accuracy, Precision, Recall, F1-score, ROC-AUC, Confusion Matrix).
2. Analyzes the baseline tree’s structure: what the root split is and why the algorithm chose it, whether the tree shows signs of overfitting, and extracts concrete decision rules.
3. Proposes and tests three improvement methods, each targeting a different weakness of the baseline:
 - **Pruning** (cost-complexity pruning) – addresses overfitting, trading tree complexity for generalization.
 - **Class-imbalance handling** (`class_weight='balanced'`, SMOTE) – addresses the model’s default bias toward the majority class (Graduate), which causes it to miss the minority classes that matter most for the application (Enrolled, Dropout).
 - **Feature selection for early warning** – addresses a practical limitation: the baseline’s strongest features are only known *after* a semester ends, so the most accurate model is not necessarily the model that is actually deployable for early intervention.
4. Quantitatively compares all five model configurations and draws conclusions not only about the numbers, but about **which model suits which application goal** – in the spirit the assignment brief emphasizes: not just running a model, but explaining how the tree is built, how it behaves, and why each improvement helps (or does not).

c. Dataset Description

c.1 Data source

The group uses the “**Predict Students’ Dropout and Academic Success**” dataset from the UCI Machine Learning Repository (ID = 697), published by Realinho, Vieira Martins, Machado, and Baptista (?), licensed CC BY 4.0. The dataset is described in detail in the accompanying paper (?).

The data was collected at **Instituto Politécnico de Portalegre**, Portugal, spanning **17 undergraduate degree programs** (Agronomy, Design, Nursing, Journalism and Communication, Management, Social Service, Biofuel Production Technologies, Informatics Engineering, etc.), academic years **2008/09 to 2018/19**. It combines information from several administrative sources at enrollment time (admission records, socio-economic data) with academic outcomes at the end of each of the first two semesters.

c.2 Dataset description — numbers confirmed from actual data

The group did not copy figures from the UCI metadata page; instead it directly inspected the raw dataset to confirm:

- The raw file contains **4,424 rows and 37 columns** $\rightarrow$ 4,424 samples (students), 36 features + 1 label column **Target**.
- There are **zero** missing values across the whole dataset (the authors pre-cleaned the data).
- **Notable finding:** the UCI page lists 36 features, but Table 1 of the original paper lists only 34. Cross-checking the actual column list against that table, the discrepancy is exactly two columns: **Previous qualification (grade)** and **Admission grade** — present in the released CSV but absent from the paper’s table. This is direct evidence the group inspected the real data rather than copying the metadata.

The 36 features fall into six groups by nature: demographics (6 columns, e.g. Gender, Age at enrollment), socio-economic status (8 columns, e.g. Tuition fees up to date, Scholarship holder), macroeconomics (3 columns: Unemployment rate, Inflation rate, GDP), academic background at enrollment (7 columns, e.g. Course, Admission grade), first-semester outcomes (6 columns), and second-semester outcomes (6 columns).

Target variable has 3 classes, moderately imbalanced:

Table 2: Target class distribution (n = 4,424)

Class	Count	%	Meaning
Graduate	2,209	49.9%	Graduated on the standard timeline (3 years; 4 for Nursing)
Dropout	1,421	32.1%	Dropped out
Enrolled	794	17.9%	Still enrolled past the standard timeline

Enrolled is the minority class and the hardest to classify, as it sits “between” the other two classes in feature space – this is the reason the class-imbalance improvement (Section f, f.2) exists.

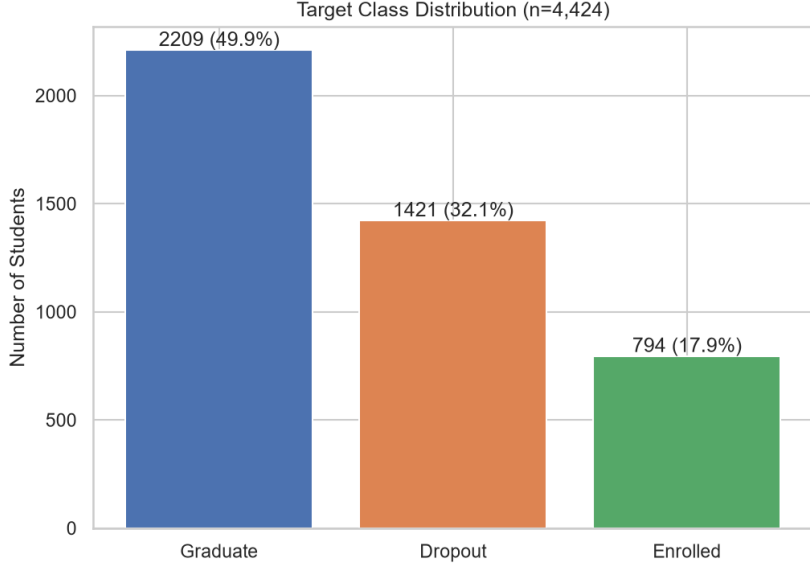


Figure 1: Target class distribution ($n = 4,424$).

Several of the observations below refer to how strongly a feature drives the baseline decision tree’s actual splits, measured by Gini importance (the total reduction in class impurity attributed to that feature across the whole tree, described further in Section d). Table 3 lists the top 10 features by this measure, plus one lower-ranked feature included for contrast with the discussion below.

Table 3: Top-10 features by Gini importance in the baseline tree (M0), plus one feature shown for contrast

Rank	Feature	Gini importance
1	Curricular units 2nd sem (approved)	0.340
2	Curricular units 2nd sem (grade)	0.045
3	Tuition fees up to date	0.045
4	Curricular units 2nd sem (enrolled)	0.045
5	Admission grade	0.044
6	Course	0.044
7	Curricular units 1st sem (evaluations)	0.034
8	Previous qualification (grade)	0.031
9	Curricular units 1st sem (approved)	0.029
10	Mother’s occupation	0.028
...		
24	Scholarship holder	0.010

Descriptive statistics from exploratory data analysis illustrate how strongly some features relate to the target: among students who had **not paid tuition on time**, 86% ended up in the Dropout class, versus 25% Dropout / 56% Graduate among students who had paid on time. Students with a **scholarship** have a $3\times$ lower Dropout rate than those without (12% vs. 39%).

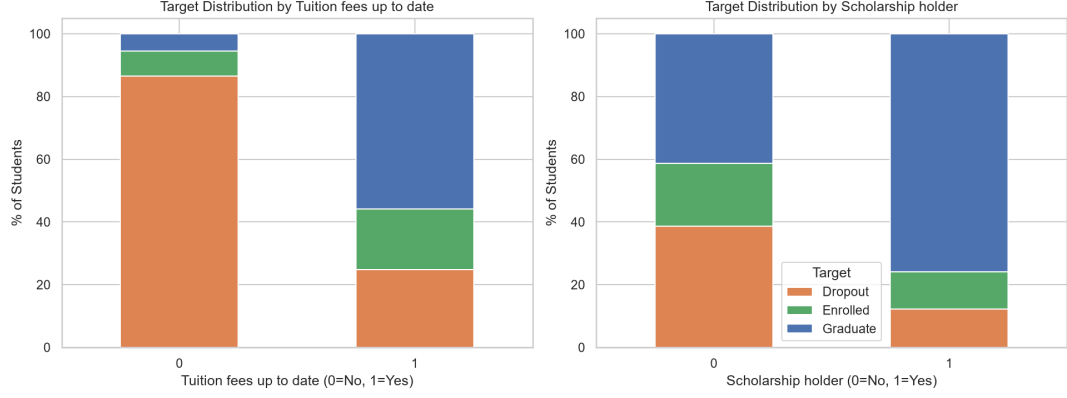


Figure 2: Target distribution split by **Tuition fees up to date** (left) and **Scholarship holder** (right).

Tuition fees up to date is among the top-5 features by permutation importance in the original paper (?), and is also rank 3 in Table 3 – the two ways of measuring importance agree here. **Scholarship holder** is a more subtle case: it is **not** in the top-5 of either the paper or Table 3 (rank 24), even though its percentage-of-target chart above looks just as striking as tuition’s. A large gap in a simple percentage-of-target chart does not automatically mean a feature is an important tree split once it is placed alongside stronger, possibly overlapping features such as **Tuition fees up to date** – the tree may be extracting the same signal from a stronger correlate instead.

By program of study, the two lowest-graduation programs are course code 33 (Biofuel Production Technologies, 66.7% Dropout) and course code 9119 (Informatics Engineering, 54.1% Dropout) – only **8.3%** and **8.2%** of their students graduate respectively, versus the dataset average of 49.9%. The two highest-graduation programs are course code 9238 (Social Service, 69.9% Graduate) and course code 9500 (Nursing, 71.5% Graduate), with the lowest Dropout rates (18.3% and 15.4%).

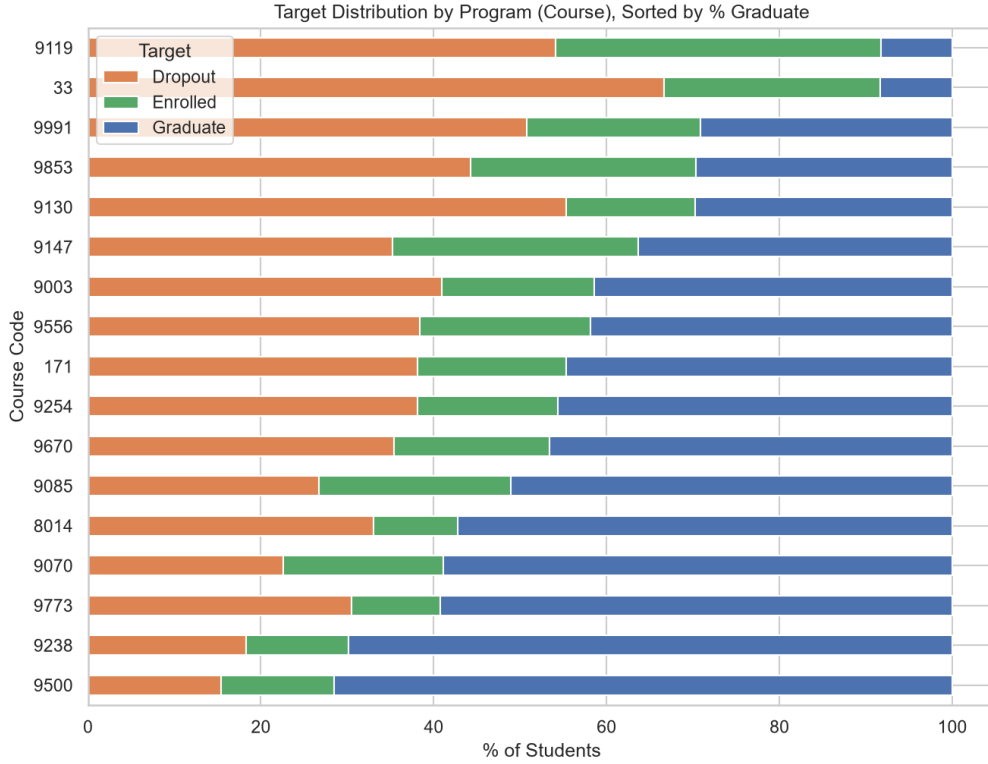


Figure 3: 17 programs sorted by increasing Graduate rate.

The original paper ranks **Course** in its top-5 features by permutation importance (?); in Table 3, **Course** ranks 6th – close to top-5 either way, and importantly, available at enrollment time, which is why it underpins the early-warning model M3 (Section f, f.3).

The strongest single predictor in the whole dataset, by contrast, is only known once a semester has actually happened:

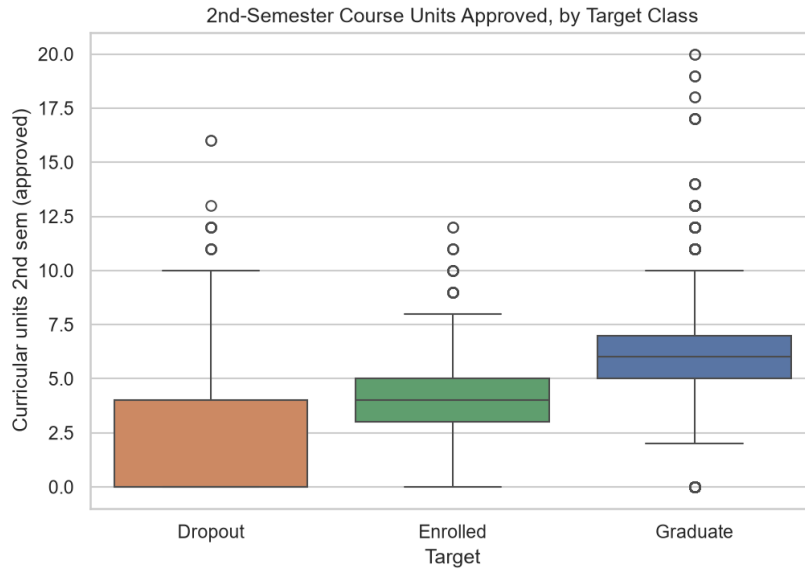


Figure 4: Curricular units 2nd sem (approved) by Target class.

Median values are 0 for Dropout, 4 for Enrolled, and 6 for Graduate. This is the single

most important feature in Table 3 (rank 1, importance 0.340 – more than the sum of ranks 2–5 combined), but it is only known *after* the second semester ends, so it cannot be used by the early-warning model M3.

c.3 Preprocessing performed

All preprocessing logic is implemented in a single shared pipeline, used by the whole team so every model is evaluated on the exact same train/test split.

1. **No missing-value handling needed** – the dataset is already clean at the source (0 missing values).
2. **Re-encoding categorical columns stored as integers.** Many columns are integer codes for unordered categories (e.g. **Course** code 33 = Biofuel Production Technologies, code 9500 = Nursing – these codes carry no ordinal relationship). Left as-is, scikit-learn would treat them as continuous and produce meaningless splits like **Course** ≤ 8.5 . The group used a mixed strategy:
 - **One-hot encode** the 4 low-cardinality columns: **Marital Status** (6), **Application mode** (18), **Course** (17), **Previous qualification** (17) – so each split reads as a clear yes/no question (e.g. “is this the Nursing program?”).
 - **Keep the raw integer code** for high-cardinality columns: the mother’s and father’s occupations (32/46 values), the mother’s and father’s qualifications (29/34 values), and **Nacionality** (21 values) – one-hot encoding these would create dozens of sparse columns, destabilizing the tree more than it would help. This trade-off is noted explicitly as a limitation.
 - After this step the feature count grows from 36 to **90** columns.
3. **No feature scaling.** A decision tree splits on a threshold of one feature at a time, not on distances between data points as KNN or linear regression do, so it is invariant to any monotonic per-column transform such as scaling. Adding **StandardScaler/MinMaxScaler** here would be redundant and would not change the learned tree structure at all.
4. **Stratified train/test split:** 80%/20%, stratified by class to preserve the $\approx 50/32/18$ class ratio in both sets – required because the data is imbalanced, to avoid a random split accidentally starving the test set of the minority class. A fixed random seed is used so all 5 models in the project are evaluated on the exact same split.

After preprocessing: **3,539 training samples / 885 test samples**, each with 90 numeric features, ready to feed directly into a decision tree classifier.

c.4 Multicollinearity among features

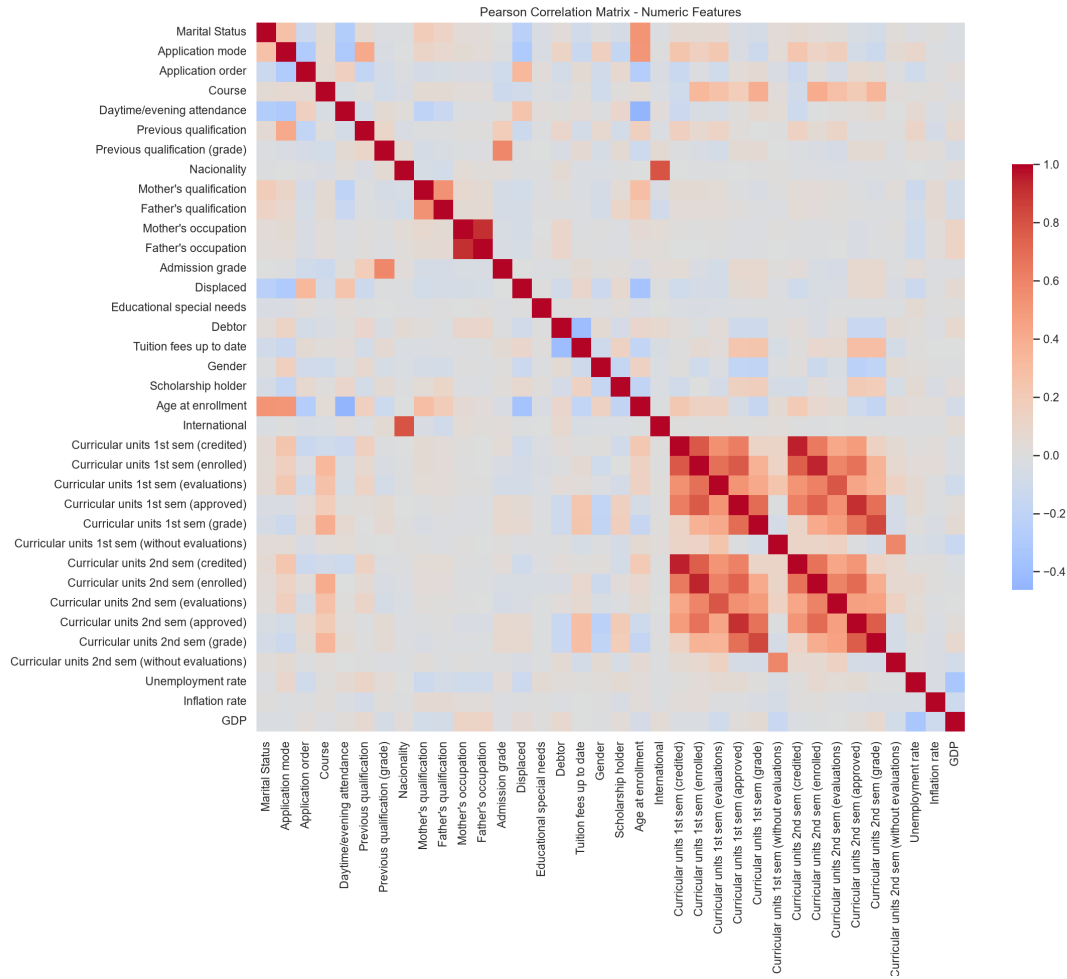


Figure 5: Pearson correlation heatmap across all numeric features.

The heatmap's colors show the overall pattern, but individual cell values are not printed on a 36×36 grid; Table 4 lists the exact values for the pairs discussed here so they can be checked directly.

Table 4: Highest Pearson correlations in the dataset

Feature	Correlated with	r
Curricular units 1st sem (credited)	Curricular units 2nd sem (credited)	0.945
Curricular units 1st sem (enrolled)	Curricular units 2nd sem (enrolled)	0.943
Mother's occupation	Father's occupation	0.910
Curricular units 1st sem (approved)	Curricular units 2nd sem (approved)	0.904
Nacionality	International	0.791

The three highest-correlation pairs in the whole dataset are `1st sem credited` $\leftrightarrow$ `2nd sem credited`, `1st sem enrolled` $\leftrightarrow$ `2nd sem enrolled`, and `Mother's occupation` $\leftrightarrow$ `Father's occupation` – this last pair is *higher* than the semester-approved pair (`1st/2nd sem approved`). The pair `Nacionality` $\leftrightarrow$ `International` is still a high correlation (international students almost always have a non-Portuguese `Nacionality` code), but lower than the four pairs above.

A decision tree is not as sensitive to multicollinearity as linear regression (it does not need redundant features removed to converge), but multicollinearity still splits feature importance across correlated columns and can make the tree structure less stable across samples. These correlations therefore provide diagnostic context, but they are not the selection rule for M3. The canonical early-warning experiment removes exactly the 12 semester-outcome columns based on availability at enrollment, while retaining both `Nacionality` and `International` so that feature timing is the only intentional difference from M0.

c.5 Why this dataset is appropriate for decision tree modeling

1. **Features have clear, real-world semantics** (tuition, scholarship, semester outcomes, program of study, ...), so every split translates into an ordinary sentence – this plays directly to a decision tree’s interpretability strength, unlike image or text data where that advantage is less exploitable.
2. **A mix of categorical and numerical features** – after nominal categories are encoded, the tree handles the resulting numerical and indicator columns without scaling. This is unlike KNN and many linear-model workflows, where feature scale directly affects distances or optimization.
3. **A genuine class imbalance** (50%/32%/18%) – creates a realistic scenario to apply and compare imbalance-handling techniques, one of the improvement directions the brief itself suggests (Section 3.2.d, “Handling class imbalance”).
4. **Features split cleanly by collection time** (known at enrollment vs. known only at end of semester) – enables a feature-selection experiment with real application meaning (early warning), not just a statistics-driven selection.
5. **No missing values** – lets the group spend its time on analysis and interpretation instead of data cleaning, matching the brief’s own emphasis: not just running a ready-made model, but clearly explaining how the tree is built and why the improvements help.

c.6 Ethical note

The dataset contains sensitive variables: gender, nationality, marital status, and parents’ education/occupation. Any tree splits on these variables only reflect **correlations observed in one Portuguese university’s sample from 2008–2019**, not causal relationships, and should not be used as grounds for any discriminatory policy toward students in practice.

d. Baseline Model

d.1 Model description and training procedure

[TODO — Describe the unconstrained Gini baseline (random seed 42) and the shared fixed, stratified split: 3,539 training rows and 885 test rows.]

d.2 Resulting tree

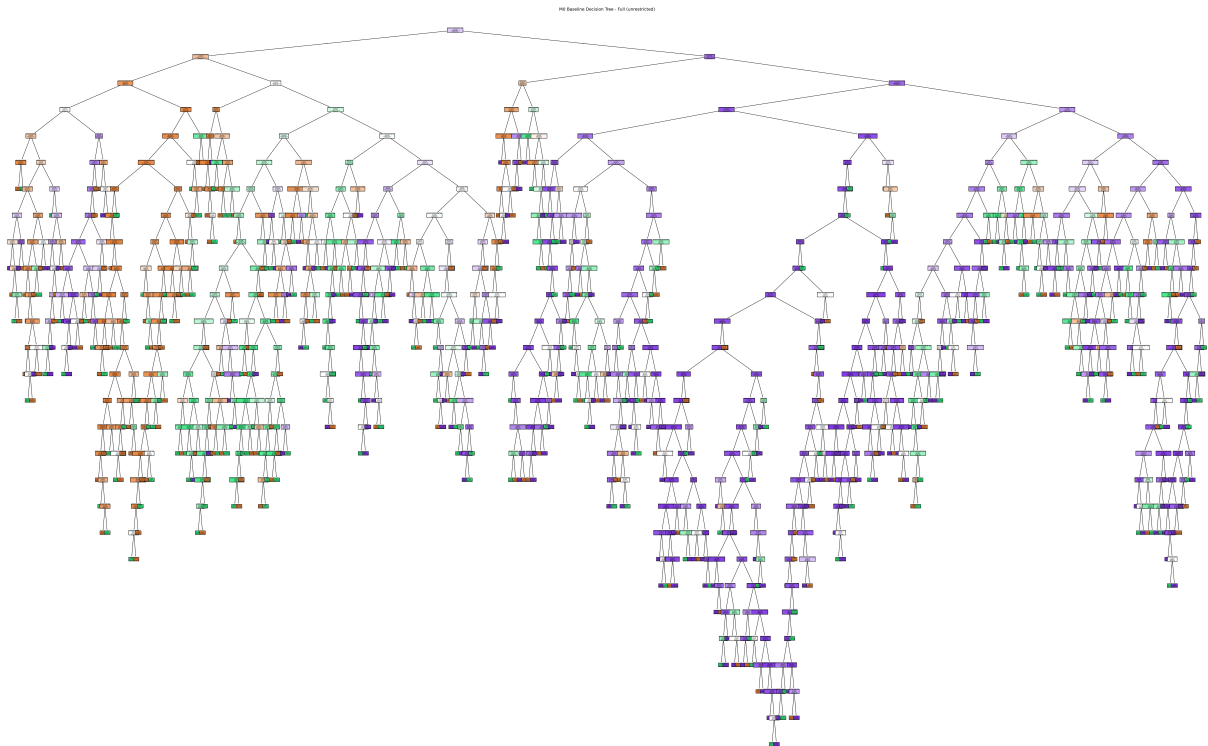


Figure 6: [TODO — Caption: full M0 tree – state node/leaf count and what it illustrates (overfitting).]

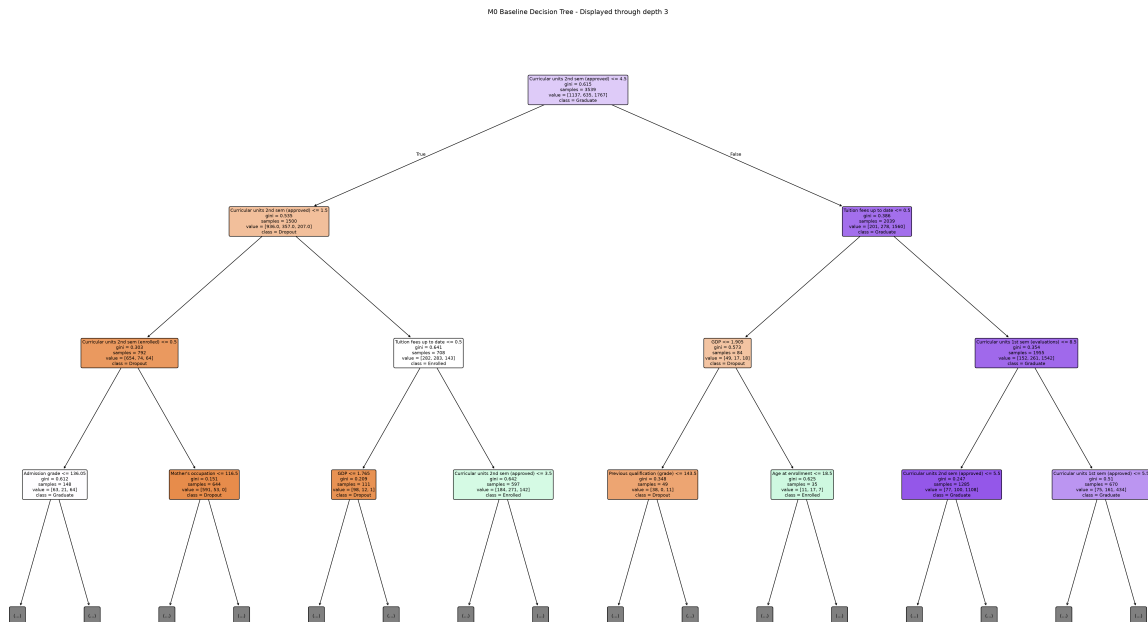


Figure 7: [TODO — Caption: M0 tree truncated to max_depth=3 for readability – describe the root and first splits shown.]

d.3 Accuracy and error rate

[TODO — Report `train_acc`, `test_acc`, `error_rate` from `outputs/results.csv` (M0 row) and `outputs/classification_report_M0.txt`. Include the confusion matrix figure below.]

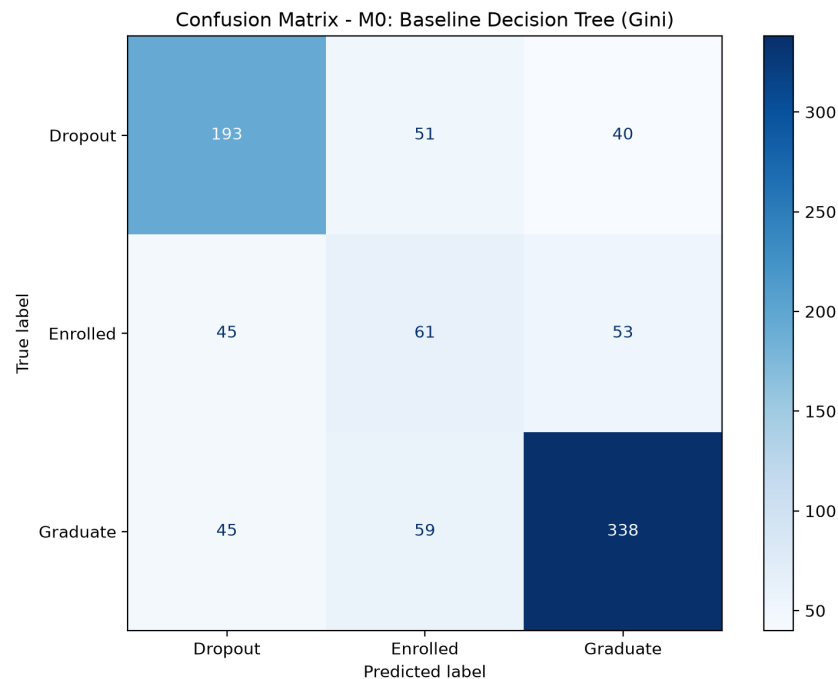


Figure 8: [TODO — Caption: confusion matrix on the 885-sample held-out test set.]

[TODO — Optional: the Gini-vs-Entropy quick check mentioned in `docs/02-DATASET-VA-CONG-VIEC.md` C0 note – one sentence is enough, this is explicitly not one of the 3 official improvements.]

e. Analysis of the Tree

e.1 Root split

[TODO — Which feature/threshold is the root split, its Gini before/after, and why the algorithm picked it over alternatives.]

e.2 Root and the next three levels

[TODO — Table or description of the first 3-4 levels: split feature, threshold, sample counts, class distribution, predicted class at each node – translate the verified node table from the source draft.]

e.3 Tree complexity

[TODO — Total node/leaf count, % of leaves with 1-2 samples, average leaf depth – and what that says about overfitting risk.]

e.4 Overfitting evidence

[TODO — Train vs. test accuracy gap (generalization gap), and whether it falls in the expected 0.65–0.72 test-accuracy band from AGENT.md (a red flag if not).]

e.5 Representative IF-THEN rules

[TODO — Translate the 3 rules from the source draft verbatim (Dropout, Graduate, Enrolled), each with the full condition chain, leaf sample count n , and purity. Do not paraphrase the conditions loosely – copy the exact feature/threshold values, they were independently re-verified against outputs/rules_M0.txt.]

Example rule format (replace with the real translated rules):

IF Curricular units 2nd sem (approved) ≤ 1.5 AND ...

THEN Dropout ($n = 189$, purity 100%).

e.6 Strengths and weaknesses of the baseline tree

[TODO — Summarize: strengths (readable top splits, handles nonlinearity, no scaling needed) vs. weaknesses (hard to present as a whole, unstable predictions, especially weak on Enrolled, uses semester-outcome features so not usable for early warning).]

[TODO — Ethical/limitations note (see source draft’s own closing section): sensitive features (gender, Nacionality, marital status, parents’ occupation/education) appear in some rule conditions – these must be flagged as correlational, not causal, and not used as grounds for discrimination.]

f. Improvement Methods

f.1 Improvement Method 1: Cost-Complexity Pruning

f.1.1 Method description

The unconstrained baseline tree exhibits the characteristic pattern of overfitting: it fits the training data perfectly, but its accuracy drops substantially on the held-out partition and its large structure is difficult to interpret. The first improvement therefore asks whether most of that structure can be removed without sacrificing predictive performance. M1 combines two complementary forms of regularization:

1. **Cost-complexity post-pruning** removes branches whose reduction in impurity is too small to justify their additional leaves. For a subtree T , the regularized objective is

$$R_\alpha(T) = R(T) + \alpha|\tilde{T}|, \tag{1}$$

where $R(T)$ is the total leaf impurity and $|\tilde{T}|$ is the number of terminal nodes. Increasing α places a larger penalty on complex trees and therefore prunes more of the weakest branches (??).

2. **Pre-pruning constraints** limit tree depth and require each leaf to contain a minimum number of training samples. These constraints prevent deep branches from forming around very small, potentially high-variance subsets of the training data.

All hyperparameter selection was performed only on the training partition. The held-out test partition was not consulted while generating the pruning path, selecting α , or searching the structural grid; it was used only after the complete M1 configuration had been finalized. Both exhaustive searches used `GridSearchCV`, and class proportions were preserved across folds with `StratifiedKFold` (??). The complete protocol is summarized in Table 5.

Table 5: Training-only model-selection protocol for M1

Stage	Procedure and outcome
Candidate generation	The cost-complexity path contained 334 points. Removing the final one-node-tree endpoint and duplicate values left 236 unique candidate values of <code>ccp_alpha</code> .
Alpha selection	Five-fold stratified cross-validation with shuffling and random seed 42; mean validation accuracy was the prespecified selection metric and macro-F1 was retained as a diagnostic metric.
Alpha tie-break	Two candidates tied on mean validation accuracy within a tolerance of 10^{-12} . The larger value was selected to prefer the simpler tree: $\alpha = 0.00148746$, with mean CV accuracy 0.747669 and mean CV macro-F1 0.672329.
Structural search	With α fixed, all 16 pairs formed by depth limits 5, 8, 10, and 15 and minimum leaf sizes 1, 5, 10, and 20 were evaluated by the same five-fold protocol.
Held-out evaluation	The selected model was refitted on the complete training partition, then evaluated on the untouched test partition. No test-set metric was used to select a hyperparameter.

f.1.2 Modified tree / model setting

Table 6 reports mean validation accuracy for every structural configuration. The values are cross-validation scores computed from training folds, not held-out test scores. Several configurations perform similarly; the selected configuration should therefore be interpreted as the best-performing candidate in this prespecified grid, not as a universally optimal setting.

Table 6: Mean cross-validation accuracy for the structural grid

max_depth	min_samples_leaf			
	1	5	10	20
5	0.741167	0.740885	0.740603	0.748235
8	0.747103	0.747386	0.747104	0.745689
10	0.747951	0.747669	0.747387	0.746536
15	0.747669	0.747669	0.747387	0.746536

The structural grid increases mean CV accuracy from 0.747669 for the alpha-only model to 0.748235, but mean CV macro-F1 decreases from 0.672329 to 0.654465. This divergence follows directly from optimizing overall accuracy rather than a class-balanced metric and foreshadows the Enrolled-recall trade-off observed on the held-out partition. The resulting M1 setting and its cross-validation scores are collected in Table 7. The fold-to-fold standard deviations caution against treating the small score differences in Table 6 as evidence that nearby configurations are meaningfully inferior.

Table 7: Selected hyperparameters and fitted complexity of M1

Quantity	Selected or observed value
ccp_alpha	0.00148746
max_depth	5
min_samples_leaf	20
Random seed	42
Mean CV accuracy	0.748235 ± 0.015124
Mean CV macro-F1	0.654465 ± 0.031242
Fitted tree depth	5
Fitted number of leaves	17

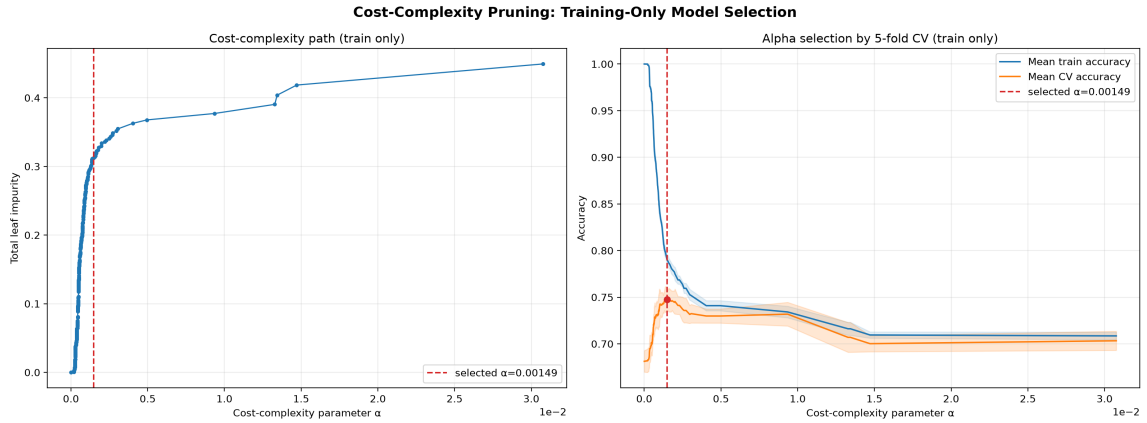


Figure 9: Cost-complexity path and cross-validation accuracy computed on the training partition.

In Figure 9, the left panel shows that total leaf impurity rises as the effective α removes progressively more structure. The right panel shows the corresponding fall in training accuracy and the validation-accuracy peak used to select α . The shaded bands show the mean score plus or minus one standard deviation across folds. No test curve is shown because the test partition was deliberately excluded from model selection.

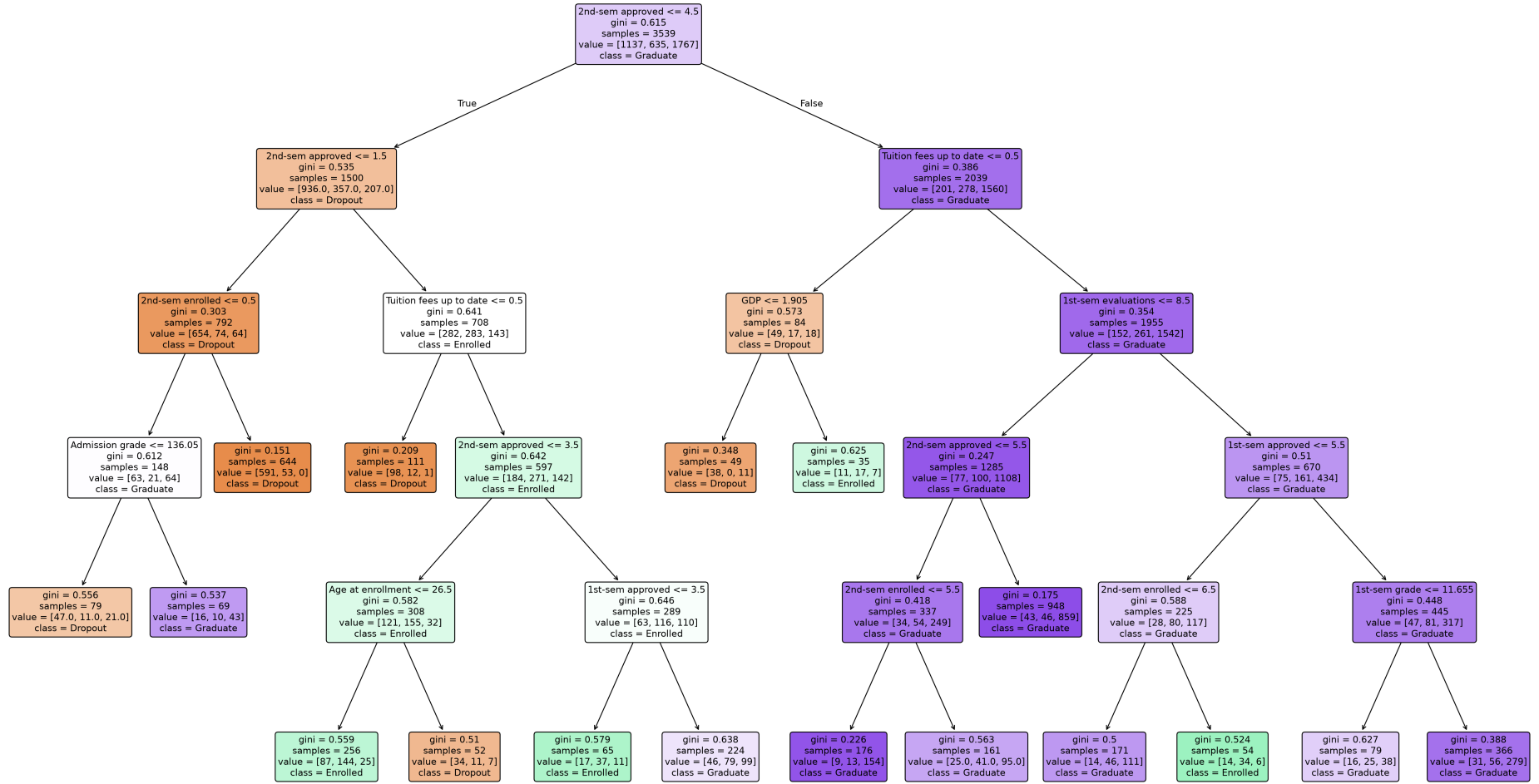
M1 - Cost-complexity pruned decision tree ($\alpha=0.001487$, depth=5, leaves=17)

Figure 10: Complete pruned M1 tree with shortened semester-feature labels for legibility.

Unlike the unconstrained baseline, Figure 10 presents the complete M1 tree at a scale that permits direct examination of its nodes and branches. Its fitted depth and leaf count are reported in Table 7.

f.1.3 Accuracy and error rate

Table 8 compares M1 with the baseline on the same held-out partition. Because error rate is defined as one minus accuracy, the 8.70-percentage-point increase in test accuracy produces an equal decrease in error rate, from 0.331073 to 0.244068. M1 also improves every aggregate metric reported here, including macro precision, macro recall, macro-F1, and macro ROC-AUC.

Table 8: Baseline and pruned-model results on the shared held-out partition

Metric	M0 baseline	M1 pruning	M1 – M0
Train accuracy	1.000000	0.768014	−0.231986
Test accuracy	0.668927	0.755932	+0.087006 (+8.70 pp)
Error rate	0.331073	0.244068	−0.087006 (−8.70 pp)
Generalization gap (train – test)	0.331073	0.012081	−0.318992
Macro precision	0.607642	0.710494	+0.102852
Macro recall	0.609310	0.660165	+0.050855
Macro-F1	0.608271	0.672925	+0.064654
Macro ROC-AUC (OvR)	0.719456	0.847692	+0.128237
Recall: Dropout	0.679577	0.686620	+0.007042
Recall: Enrolled	0.383648	0.345912	−0.037736
Recall: Graduate	0.764706	0.947964	+0.183258
Tree depth	27	5	−22 (−81.48%)
Number of leaves	634	17	−617 (−97.32%)

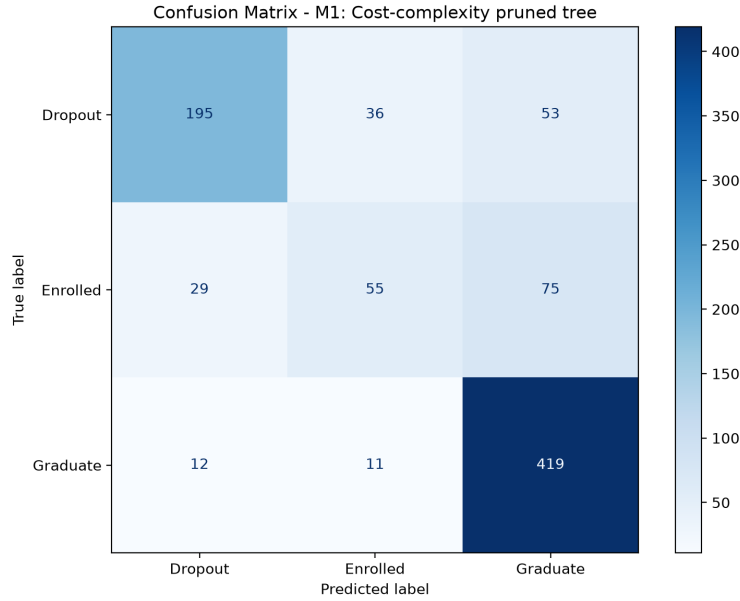


Figure 11: Confusion matrix of M1 on the held-out test partition.

Figure 11 provides the class-level counts behind the aggregate scores. M1 correctly classifies 195 of 284 Dropout students, 55 of 159 Enrolled students, and 419 of 442 Graduate students. For the Enrolled class, 29 cases are predicted as Dropout and 75 as Graduate. These counts agree with the class recalls reported in Table 8.

f.1.4 Why this improves the result

The observed pattern is consistent with the bias-variance effect expected from pruning. M0 attains perfect training accuracy by continuing to partition increasingly specific subsets of the training data. In contrast, M1 accepts higher training error in exchange for stronger regularization and a much smaller tree, which is expected to reduce variance. On the held-out split, test accuracy increases while the train-test accuracy gap decreases from 0.331073 to 0.012081; depth and leaf count also fall by 81.48% and 97.32%, respectively. Taken together, these results suggest that much of the removed structure captured sample-specific variation rather than stable predictive signal. This interpretation is supported by the current experiment but should not be generalized to every future sample without repeated evaluation.

Performance gains are not uniform across classes. Graduate recall increases from 0.764706 to 0.947964, and Dropout recall increases slightly from 0.679577 to 0.686620, whereas Enrolled recall decreases from 0.383648 to 0.345912. Because hyperparameter selection optimized overall accuracy rather than a class-balanced metric, the procedure did not explicitly protect Enrolled recall and could therefore favor the majority Graduate class. The confusion matrix supports this interpretation: 75 of the 104 misclassified Enrolled cases are assigned to Graduate. This class-specific degradation is an important trade-off and motivates the class-imbalance experiments in the next improvement method.

Finally, the observed improvement is based on one fixed held-out split and does not imply that M1 will gain exactly 8.70 percentage points on every future sample. The fold-to-fold variation in Table 7 and the close scores in Table 6 further caution against overinterpreting the exact hyperparameter choice. For this fitted experiment, however, two conclusions are directly supported: all hyperparameters were selected without consulting the held-out test partition, and M1 improved held-out aggregate performance while substantially reducing structural complexity, with a clear Enrolled-recall trade-off. Repeated evaluation would be needed to assess the stability of the exact performance gain.

f.2 Improvement Method 2: Class Imbalance

f.2.1 Method description

[TODO — Describe both configurations: M2a = `class_weight='balanced'`; M2b = SMOTE (??), fit *only* on the training split obtained from `src.data.get_train_test()`, never before the split.]

f.2.2 Modified tree / model setting

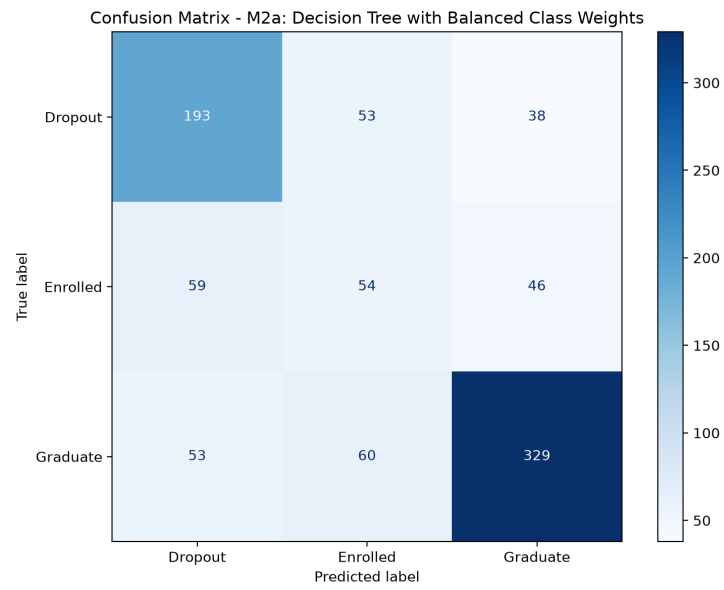


Figure 12: [TODO — Caption: M2a confusion matrix.]

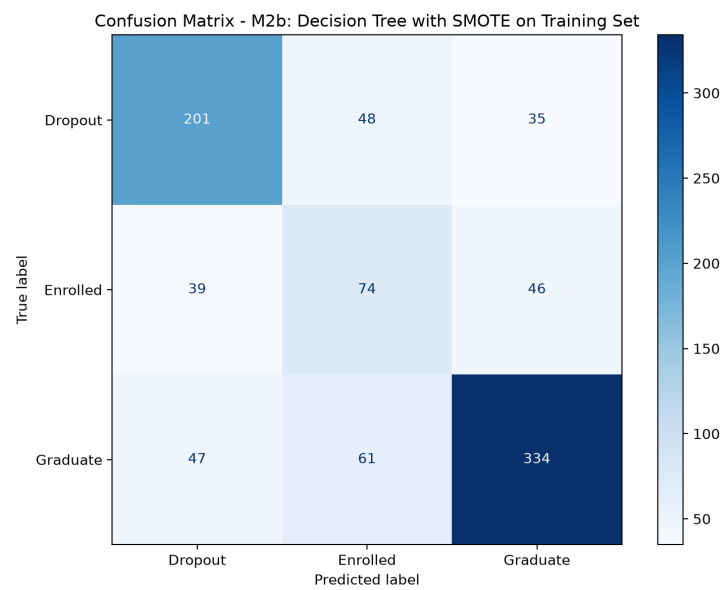


Figure 13: [TODO — Caption: M2b confusion matrix.]

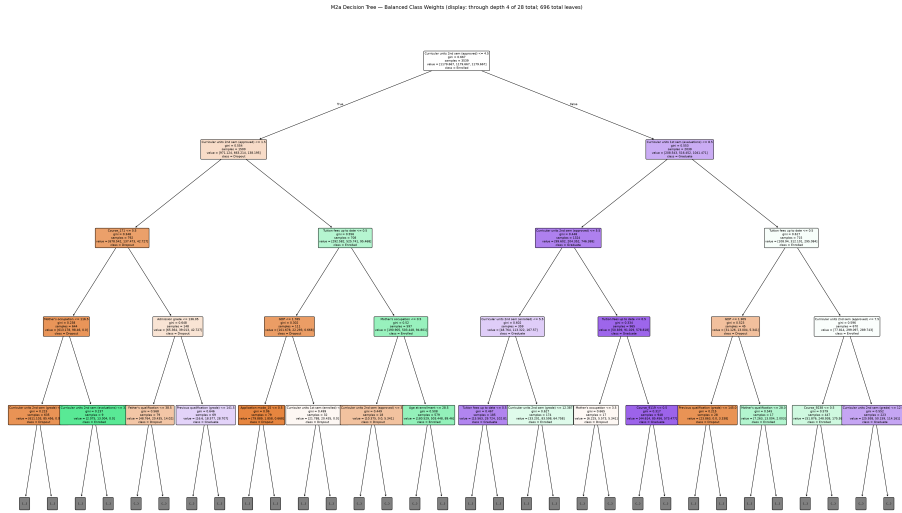


Figure 14: [TODO — Caption: M2a tree, truncated view (state real depth/leaves and note this view's depth limit).]

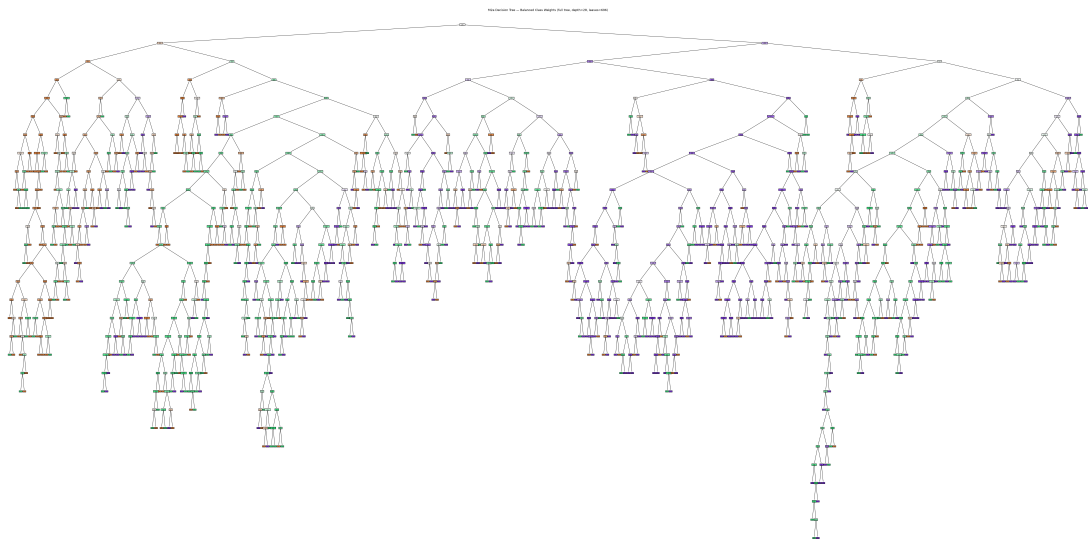


Figure 15: [TODO — Caption: full M2a tree – state it is roughly as complex as M0, since class_weight changes split priorities, not tree size.]

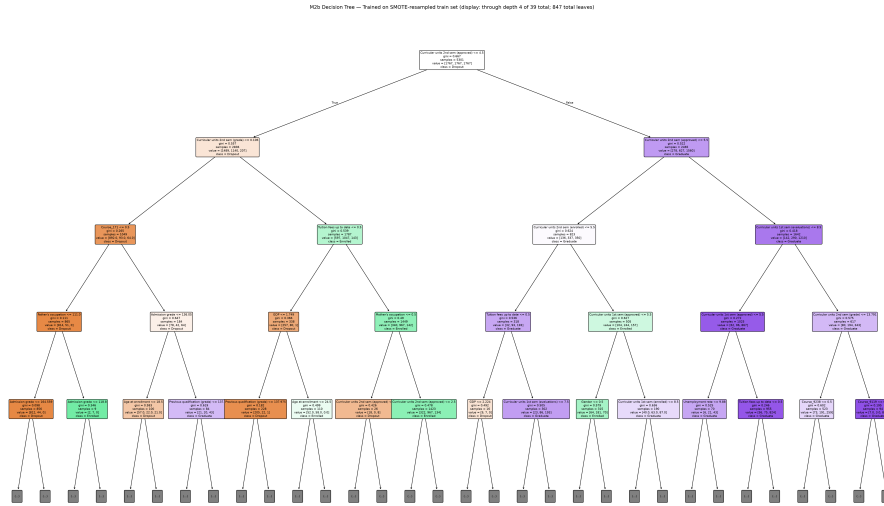


Figure 16: [TODO — Caption: M2b tree, truncated view – note it was trained on the SMOTE-resampled train set but evaluated on the original test set.]

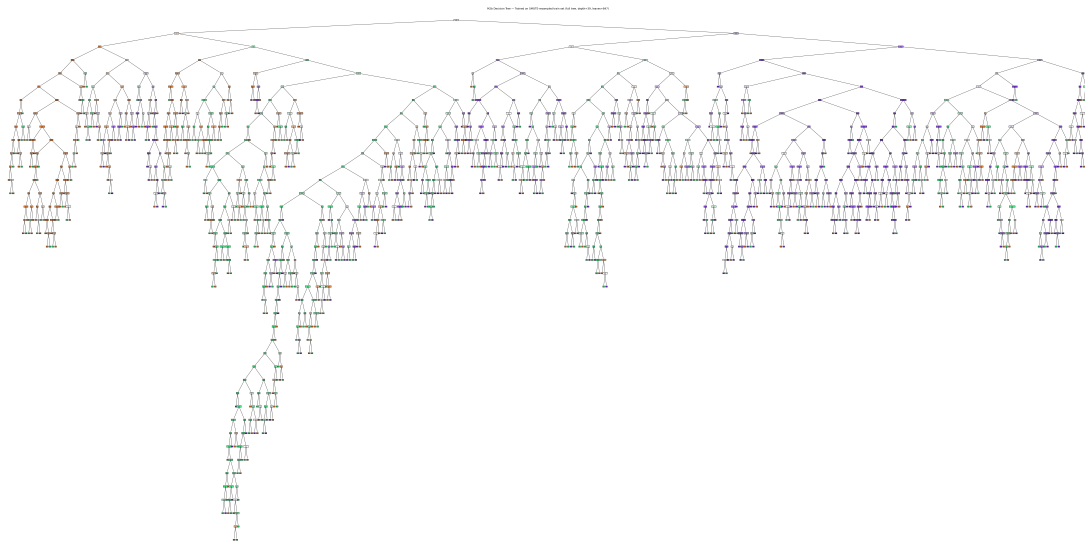


Figure 17: [TODO — Caption: full M2b tree – larger than M0/M2a because SMOTE grows the training set (3,539 → 5,301 rows), giving the tree more rows to split on.]

f.2.3 Accuracy and error rate

[TODO — Report M2a and M2b test accuracy and error rate against M0. Add the per-class recall table for Dropout, Enrolled, and Graduate across M0/M2a/M2b; that table is the point of this experiment, not overall accuracy alone.]

f.2.4 Why this does (or does not) improve the result

[TODO — State plainly that M2a does *not* help on this split (both accuracy and Enrolled recall drop) – do not hide this. Explain M2b’s improvement mechanism (synthetic minority-class samples give the tree more opportunities to find a separating split), and be explicit that recall Enrolled/Dropout going up while overall accuracy is a secondary concern is the correct way to judge success here, per

AGENT.md’s own guidance that overall accuracy is not the target metric for this improvement.]

[TODO — Optional verified limitation: vanilla SMOTE interpolates one-hot dummy vectors. Integer restoration truncates fractional values and can create invalid all-zero groups; Course was worst at 83.0%. Explain that this limitation does not invalidate metrics measured on the real held-out test set.]

f.3 Improvement Method 3: Feature Selection for Early Warning

f.3.1 Method description

[TODO — Describe the method: remove all six first-semester and all six second-semester outcome columns, retain 24 original features including the three macroeconomic variables known at enrollment, and train the same unconstrained Gini tree with seed 42. Feature availability must be the only intentional difference from M0.]

f.3.2 Modified tree / model setting

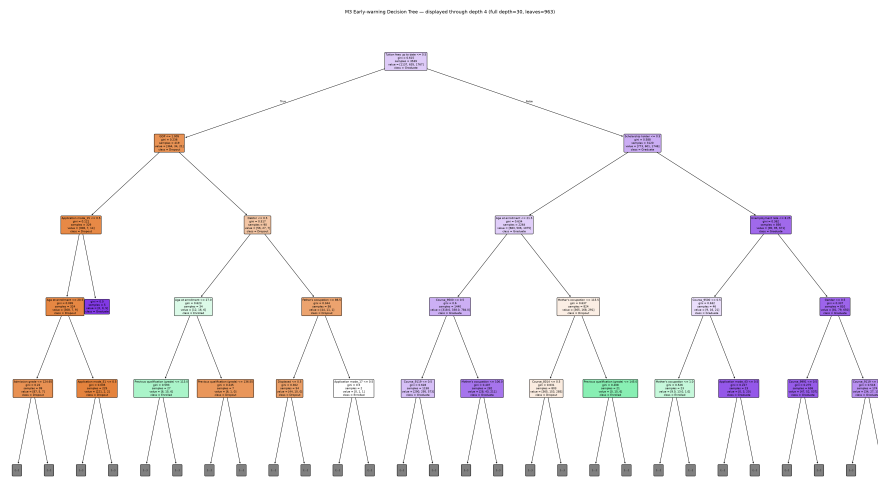


Figure 18: [TODO — Caption: M3 tree; state its full depth and leaf count.]

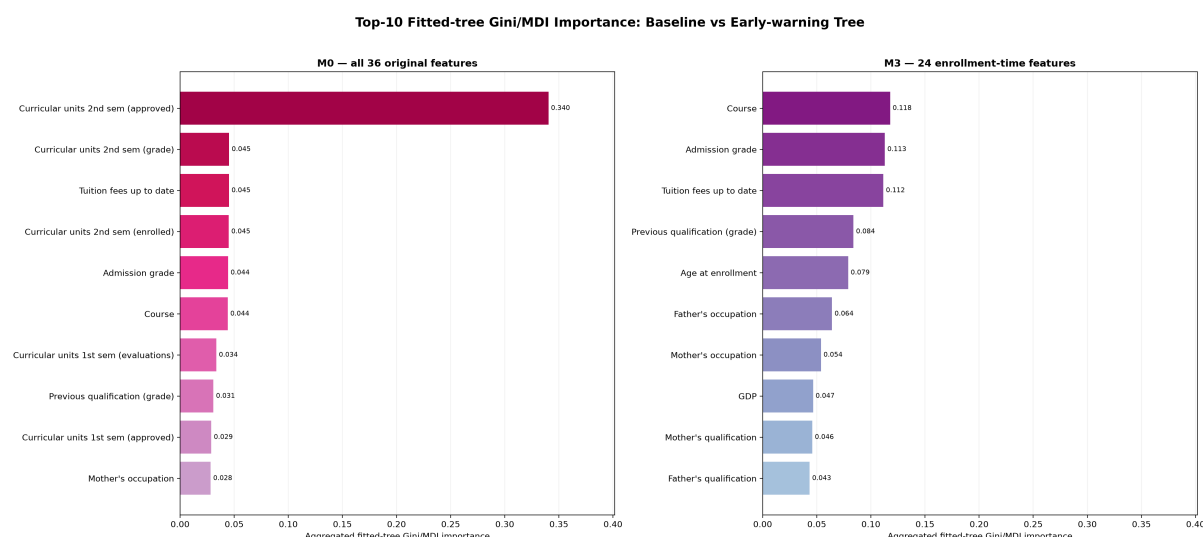


Figure 19: [TODO — Caption: top-10 Gini feature importance, M0 (all 36 original features) vs. M3 (24 retained features) side by side, one-hot dummy importances summed back to their original UCI feature name for a fair comparison.]

f.3.3 Accuracy and error rate

[TODO — Report M3 test accuracy, error rate, and macro-F1 against M0, then discuss the confusion matrix below.]

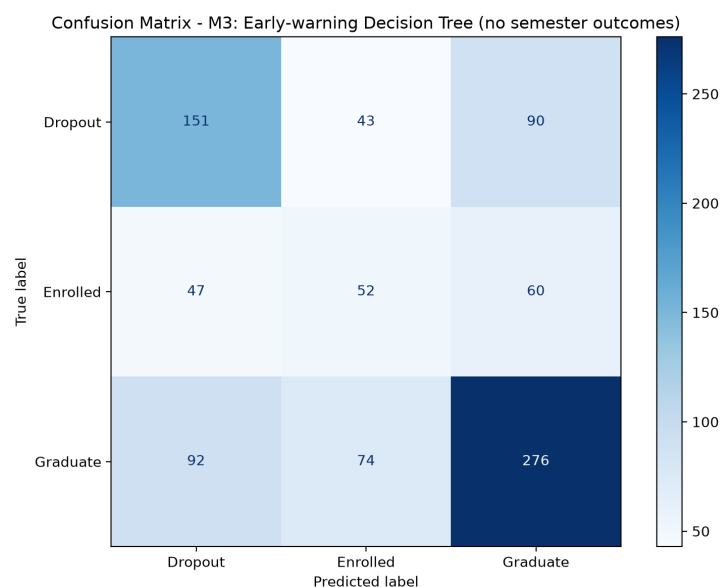


Figure 20: [TODO — Caption: M3 confusion matrix on the same 885-sample held-out test set as every other model.]

f.3.4 Why this does not improve accuracy — and why that is expected

[TODO — Explain plainly that the accuracy drop is intended, not a failure: the removed columns include several of M0's strongest Gini-importance features, especially second-semester approvals, but those values arrive too late for enrollment-time action. State the M0/M3 trade-off and cite (?) if it supports the early-prediction framing.]

g. Comparison of Results

g.1 Overall comparison table

Table 9: All 5 models on the shared 885-sample held-out test set

Model	Test acc	Error	F1 macro	ROC-AUC	Rec. Dropout	Rec. Enrolled	Rec. Graduate	Depth	Leaves
M0 (baseline)	0.6689	0.3311	0.6083	0.7195	0.6796	0.3836	0.7647	27	634
M1 (pruning)	0.7559	0.2441	0.6729	0.8477	0.6866	0.3459	0.9480	5	17
M2a (class_weight)	0.6508	0.3492	0.5854	0.7053	0.6796	0.3396	0.7443	28	696
M2b (SMOTE)	0.6881	0.3119	0.6387	0.7421	0.7077	0.4654	0.7557	39	847
M3 (early warning)	0.5412	0.4588	0.4931	0.6254	0.5317	0.3270	0.6244	30	963

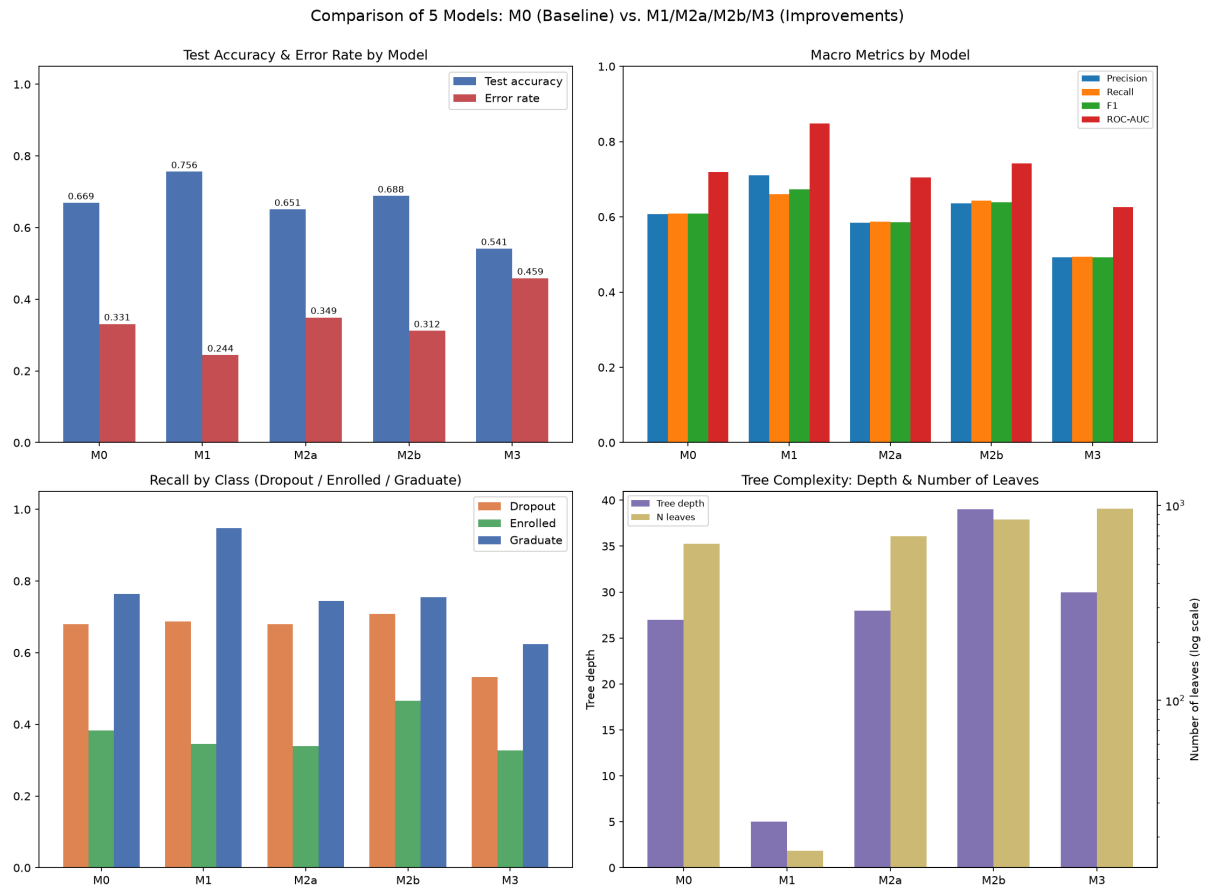


Figure 21: Four-panel comparison of M0-M1-M2a-M2b-M3 on the same held-out test set (885 samples).

Reading the four panels: top-left (test accuracy and error rate) shows M1 highest and M3 lowest; top-right (the four macro metrics – Precision/Recall/F1/ROC-AUC) shows M1 leading on all four; bottom-left (per-class recall) shows M2b dominating on Dropout and Enrolled while M1 dominates on Graduate; bottom-right (tree complexity, leaf count on a log scale) shows M1 far more compact than the other four models.

g.2 Comparing each improvement against the baseline

M1 (Pruning) improves almost every aggregate metric. Test accuracy rises from 0.6689 to 0.7559 (+8.70 percentage points), error rate drops from 33.11% to 24.41%, F1 macro

risers 6.47 points, and ROC-AUC macro rises 12.82 points. At the same time, complexity drops sharply: depth 27→5, leaves 634→17 (**about 37× fewer**). This is a favorable result: aggregate performance and structural simplicity both improve, although Enrolled recall decreases slightly (0.3836→0.3459, −3.77 points). Accuracy was the prespecified CV criterion for selecting `ccp_alpha/max_depth/min_samples_leaf`; because that criterion does not explicitly protect minority-class recall, it can favor the majority Graduate class whenever doing so raises the total number of correct predictions.

M2 (Class Imbalance) gives two opposite results, consistent with the experimental nature of this improvement. M2a (`class_weight='balanced'`) **does not improve** M0 on this test set: test accuracy drops 1.81 points, and even Enrolled recall – the metric this technique targets – also drops (0.3836→0.3396). In contrast, M2b (SMOTE on train only) improves both overall accuracy (+1.92 points) and the recall of the two hardest classes: Dropout recall +2.82 points, Enrolled recall +**8.18 percentage points** (61/159 → 74/159 Enrolled students correctly identified), at the cost of a slight 0.90-point drop in Graduate recall. M2b also raises Enrolled precision relative to M0 (0.3567→0.4044), meaning the model is not simply “guessing Enrolled more often” to buy recall – this is a genuine improvement, not an artifact of biased prediction.

M3 (Early Warning) drops performance substantially, and this is the expected result, not a failure. Test accuracy falls from 0.6689 to 0.5412 (−12.77 percentage points), F1 macro falls 11.52 points. The direct cause, measured from M0’s own tree (not inferred from the original paper): by M0’s real Gini feature importance (computed by Role E), 3 of the top 5 features are **2nd sem approved** (rank 1, importance 0.340 – over a third of the whole tree’s total importance), **2nd sem grade** (rank 2, 0.045), and **2nd sem enrolled** (rank 4, 0.045) – all three fall inside the 12 columns dropped in M3 because they are only known *after* the second semester ends. The other two features in M0’s real top-5 – **Tuition fees up to date** (rank 3) and **Admission grade** (rank 5) – are retained in M3 since they are known at enrollment, but that is not enough to make up for losing **2nd sem approved**, which alone accounts for more importance than the next four features combined. M3 must predict using only information available at enrollment time (program of study, admission grade, tuition status, the 3 macroeconomic variables, ...).

The original paper ranks first-semester approvals second using permutation importance from four ensemble algorithms (?). That is a different method from this project’s M0 Gini importance. The feature ranks ninth in M0 (importance 0.029), so the different rankings are expected. The paragraph above uses only importance values measured from M0 itself.

g.3 Which model is “best”?

There is no single answer – it depends on the criterion:

Table 10: Best model by criterion

Criterion	Best model	Why
Overall accuracy / F1 / ROC-AUC	M1	Highest on nearly every aggregate metric, and the most compact tree
Detecting Dropout students	M2b	Highest Dropout recall (0.7077)
Detecting Enrolled students (hardest class)	M2b	Highest Enrolled recall (0.4654), ~1.21× M0
Interpretability / presentation	M1	Only 17 leaves, fully readable, vs. 634–963 for the other models
Early deployment (before semester results exist)	M3	The only model that needs no semester-1/2 data

On accuracy alone, M1 wins. But the main lesson of comparing all five model configurations is that **“best” depends on the application goal**. M1 is best for an accurate, interpretable classification system used once semester data already exists. M2b is best if the goal is to miss as few at-risk students as possible (Dropout/Enrolled), accepting a small overall-accuracy trade-off. M3 is the only feasible choice if the goal is **early intervention right at enrollment** – none of the other four configurations can run at that point in time, no matter how much higher their accuracy is.

h. Conclusion

h.1 What the group learned

The project directly demonstrates the two theoretical points raised in the Introduction: a decision tree is both interpretable and prone to overfitting if its complexity is not controlled. After nominal categories are encoded, it also needs no feature scaling. The unconstrained M0 model reaches perfect training accuracy (1.000) while test accuracy is only 0.6689 – a generalization gap of 0.3311 that is direct, not inferred, evidence of the overfitting phenomenon described in the theory (??).

The three improvement directions highlight distinct lessons:

1. **For the severely overfit baseline in this project, controlling complexity is highly effective.** Pruning improves aggregate held-out performance and structural simplicity, although its effect is not uniform across classes because Enrolled recall decreases slightly.
2. **Class-imbalance techniques do not automatically help** – effectiveness depends on the specific mechanism (M2a reweights the Gini computation without adding data, while M2b adds synthetic samples in the minority class’s feature region), and must be judged by per-class recall, not overall accuracy alone – a model can be “worse” on accuracy yet “better” on the real application goal.
3. **The most accurate model is not always the most useful one.** M3 shows that this dataset’s strongest predictive features (semester outcomes) are only available once it is already too late for preventive intervention – so M3’s lower accuracy is a reasonable price for early-warning capability.

h.2 Key findings from the experiments

- M0’s overfitting: a 0.3311 generalization gap (train 1.000 vs. test 0.6689), depth 27, 634 leaves – the tree “memorizes” rather than generalizes.
- Pruning shrinks the leaf count by $\sim 37\times$ ($634 \rightarrow 17$) while raising test accuracy by 8.70 percentage points, consistent with much of the removed structure capturing sample-specific variation rather than stable predictive signal.
- `class_weight='balanced'` (M2a) does not improve minority-class recall on this data; SMOTE-on-train (M2b) raises Enrolled recall by 8.18 points and Dropout recall by 2.82 points, while also raising precision – a genuine improvement, not an artifact.
- Dropping the 12 semester-outcome columns (M3) costs 12.77 accuracy points – a direct measurement of the “price” of wanting an early warning instead of a later but more accurate one.
- All 5 models share one fixed, stratified train/test split (`random_state=42`). This makes the comparisons directly comparable and prevents different partitions from confounding the reported differences, but it does not eliminate sampling uncertainty.

h.3 Effectiveness of decision trees for this problem

A decision tree suits this dataset for the reasons argued in the Dataset Description section: features have clear semantics, so every decision rule is interpretable – for example, the Dropout rule extracted from the M0 tree in Section e is a chain of 9 conditions that together identify a leaf of $n = 189$ training samples at 100% purity; the single most readable condition in that chain is passing at most 1.5 second-semester course units (`2nd sem approved  $\leq 1.5$`), but that threshold alone is not sufficient on its own – it is one term in the full conjunction. The parallel Graduate leaf ($n = 176$, 100% purity) is a 16-condition chain, the two most readable conditions being passing more than 5.5 second-semester course units *and* `Tuition fees up to date > 0.5` – the latter consistent with the EDA observation in Figure 2 that paying tuition on time correlates strongly with graduating. Long condition chains like these are themselves a sign of the overfitting discussed above: a rule that needs 9–16 ANDed conditions to stay 100% pure is fitting narrow pockets of the training set, not a simple general pattern – one of the concrete costs of leaving M0 unpruned. After preprocessing, the tree can split numerical and one-hot indicator columns without feature scaling. But the project also exposes the inherent limits of a single, unconstrained tree: it overfits easily, and no single model configuration is simultaneously the most accurate, the fairest across classes, and the earliest-usable – these three goals require three different configurations of the same algorithm. For an application with multiple distinct goals, like dropout prediction, a more realistic conclusion than “one best model” is: **a decision tree is a flexible enough tool to be optimized for each goal separately** (accuracy, class fairness, or timeliness), provided the user chooses the right configuration and understands the trade-off being accepted – exactly the spirit the assignment brief asks for: not just running a model, but explaining why each choice does or does not help.